WanderMate: Social Mobile Travel Planner

Project 3 — Task 1 Deliverable
Requirements & Subsystem Overview

Contents

1 Functional Requirements **3**
- 1.1 Trip Itinerary Management . . . 3
- 1.2 POI Discovery & Place Details . . . 3
- 1.3 Route Visualisation . . . 4
- 1.4 Budget Tracking . . . 4
- 1.5 Social Feed & Community . . . 5
- 1.6 Collaborative Editing & Authentication . . . 6

2 Non-Functional Requirements **6**
- 2.1 Performance . . . 6
- 2.2 Scalability . . . 7
- 2.3 Availability & Reliability . . . 8
- 2.4 Security . . . 8
- 2.5 Usability & Maintainability . . . 9

3 Architecturally Significant Requirements — Summary **9**

4 Subsystem Overview **10**
- 4.1 Subsystem Summary Table . . . 11
- 4.2 Detailed Subsystem Descriptions . . . 12
  - 4.2.1 Mobile Client (React Native / Expo) . . . 12
  - 4.2.2 API Gateway (Node.js + Express) . . . 13
  - 4.2.3 POI & Routing Service (TravelService Facade) . . . 13
  - 4.2.4 Itinerary & Budget Engine . . . 13
  - 4.2.5 Social Feed & Community Service . . . 14
  - 4.2.6 Authentication & User Profile . . . 14
  - 4.2.7 Real-time Sync Infrastructure (Firebase — Cross-cutting) . . . 14

5 Subsystem Interaction Flows **15**

1. Functional Requirements

Functional requirements define what the WanderMate system must *do* — the behaviours and capabilities that directly serve user goals. The 23 requirements below are derived from the six core feature areas identified in the project proposal. Each is assigned a unique identifier, a category, a description, and a priority level (**High** / Medium).

High	Critical path; must be implemented in the prototype.
Medium	Important but deferrable to the final release.

1.1 Trip Itinerary Management

> **Architectural significance.** The itinerary is the central domain entity that every other subsystem reads from or writes to. Its schema design drives the MongoDB data model, the Firebase Realtime Database structure, the offline-storage strategy, and the Command-pattern undo/redo stack.

Req ID	Category	Requirement	Priority
FR-01	Itinerary	Users shall be able to create a new trip with a name, destination, and date range.	**High**
FR-02	Itinerary	Users shall be able to add, edit, remove, and reorder stops within any day of a trip via drag-and-drop.	**High**
FR-03	Itinerary	The system shall persist itinerary data locally (AsyncStorage) and synchronise to the backend when connectivity is restored.	**High**
FR-04	Itinerary	Users shall be able to perform undo and redo on itinerary editing actions (add stop, remove day, reorder).	Medium

1.2 POI Discovery & Place Details

> **Architectural significance.** The dual-API design (Overpass as primary, Google Places for enrichment) introduces a Strategy pattern abstraction and a caching layer to control free-tier quota consumption. Switching providers must require zero changes to UI components.

Req ID	Category	Requirement	Priority
FR-05	POI Discovery	Users shall be able to search for hotels, restaurants, and landmarks by keyword and category using the Overpass API (OpenStreetMap data).	**High**
FR-06	POI Discovery	The system shall display ratings, photos, and opening hours for each POI via the Google Places API.	**High**
FR-07	POI Discovery	API responses from Overpass and Google Places shall be cached to reduce latency and respect free-tier quotas (Google Places: 5 000 req/month).	**High**
FR-08	POI Discovery	Users shall be able to add a discovered POI to any day of their active itinerary with a single tap.	**High**

1.3 Route Visualisation

> **Architectural significance.** Embedding Leaflet inside a React Native WebView creates a cross-boundary JavaScript bridge. Route optimisation calls OpenRouteService (ORS) and must respect its daily quota of 2 000 requests, necessitating caching and client-side debouncing.

Req ID	Category	Requirement	Priority
FR-09	Routing	The system shall render an interactive map of each day's itinerary using Leaflet and OpenStreetMap tiles inside a React Native WebView.	**High**
FR-10	Routing	The system shall compute an optimised multi-stop route for each itinerary day via the OpenRouteService API and overlay the polyline on the map.	**High**
FR-11	Routing	The map shall update in real time as stops are added, removed, or reordered in the itinerary editor, without requiring a manual refresh.	Medium

1.4 Budget Tracking

> **Architectural significance.** Expense data must be associated with trips and users, influencing the MongoDB schema with nested subdocuments. Split-expense calculation requires deterministic server-side logic to remain consistent across all collaborators on a shared trip.

Req ID	Category	Requirement	Priority
FR-12	Budget	Users shall be able to log individual expenses, each tagged with a category (accommodation, food, transport, activities, other) and a specific itinerary day.	**High**
FR-13	Budget	The system shall display per-day expense summaries and a running trip total, automatically updating as new expenses are added or removed.	**High**
FR-14	Budget	Users shall be able to split a group trip's total expenses equally among participants and view each person's calculated share.	Medium
FR-15	Budget	Budget data shall synchronise across all collaborators on a shared trip in real time via Firebase Realtime Database.	Medium

1.5 Social Feed & Community

> **Architectural significance.** The social feed introduces a publish/subscribe model at the data layer that is distinct from the collaborative editing channel. Feed post volume could grow unbounded, requiring cursor-based pagination in MongoDB and indexed queries in Firebase to avoid full-collection scans.

Req ID	Category	Requirement	Priority
FR-16	Social Feed	Users shall be able to publish a completed trip to a public social feed; each post shall display a cover thumbnail, trip duration, participant count, and total budget summary.	**High**
FR-17	Social Feed	Users shall be able to follow other travellers and browse a personalised feed of published itineraries from people they follow.	**High**
FR-18	Social Feed	Users shall be able to like published itineraries; like counts shall be visible on feed cards and update in real time.	Medium
FR-19	Social Feed	Users shall be able to clone any public itinerary into their own trip list as an independently editable copy with a single tap.	**High**

1.6 Collaborative Editing & Authentication

> **Architectural significance.** Real-time collaborative editing requires conflict-free state propagation (Observer pattern via Firebase listeners). Shared write access to trip documents demands authenticated session management and Firebase Security Rules enforced at the database layer to prevent unauthorised mutations.

Req ID	Category	Requirement	Priority
FR-20	Collaboration	Users shall be able to invite other registered users to co-edit a trip. Changes by any collaborator shall propagate to all others in real time via Firebase Realtime Database.	**High**
FR-21	Collaboratio	The system shall resolve concurrent edits using last-write-wins semantics; all mutations shall be timestamped server-side to establish a total order.	**High**
FR-22	Auth	Users shall be able to register and sign in using Google Sign-In via Firebase Authentication (OAuth 2.0).	**High**
FR-23	Auth	Authenticated sessions shall persist across app restarts and device reboots until the user explicitly signs out.	Medium

2. Non-Functional Requirements

Non-functional requirements (NFRs) govern the quality attributes of WanderMate beyond feature correctness. Each NFR below is architecturally significant — it has a direct impact on technology selection, pattern adoption, or structural decomposition, and each is accompanied by a concrete, measurable target.

2.1 Performance

> **Architectural impact.** NFR-01 and NFR-02 drive the caching layer for Overpass and Google Places API responses inside the TravelService Facade. NFR-03 justifies the event-driven Firebase approach over a REST polling model, which would introduce noticeable lag for collaborators and unnecessary bandwidth consumption.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-01	Performance	The app shall render the itinerary map and return POI search results within 2 seconds on a 4G mobile connection.	$< 2\,\mathrm{s}$ at p95
NFR-02	Performance	The social feed shall paginate results and display the first page of cards within 1.5 seconds of the Feed tab being opened.	$< 1.5\,\mathrm{s}$ first-page load
NFR-03	Performance	Real-time itinerary updates shall propagate to all active collaborators within 500 ms of a change being committed to Firebase.	$< 500\,\mathrm{ms}$ sync latency

2.2 Scalability

> **Architectural impact.** NFR-06 is the primary economic constraint of the entire architecture. It is the central driver of the TravelService Facade (to centralise quota control) and the Repository caching strategy (to minimise outbound API calls). Exceeding free-tier limits would introduce direct financial cost to the project at launch.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-04	Scalability	The Node.js + Express API Gateway shall support at least 200 concurrent users without degradation in response time beyond the targets set in NFR-01 and NFR-02.	200 concurrent users
NFR-05	Scalability	The MongoDB data schema shall support horizontal sharding by user ID to accommodate growth beyond 10 000 registered users without schema migration.	10 000+ users
NFR-06	Scalability	The system shall operate entirely within free-tier API quotas at launch: Overpass (unlimited), Google Places ($\leq$ 5 000 req/month), ORS ($\leq$ 2 000 req/day), achieved through caching and per-user rate limiting.	Zero overage cost at launch

2.3 Availability & Reliability

> **Architectural impact.** NFR-07 mandates AsyncStorage-backed offline-first storage and a persistent sync queue in the client layer. NFR-08 reinforces the Strategy pattern for POI providers so that the active provider can be swapped at runtime without restarting the application or impacting the UI layer.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-07	Availability	Core itinerary viewing and editing shall remain available when the device is offline. Changes shall be queued locally and synchronised to the server upon reconnection, with no data loss.	100% offline read/write for cached data
NFR-08	Reliability	If the Google Places API is unavailable, POI search shall degrade gracefully to Overpass-only results; no user-facing crash or blank screen shall occur.	Zero crashes on external API failure
NFR-09	Reliability	The API Gateway shall return a structured error response (HTTP 4xx/5xx with a JSON error body) for every failure scenario; no unhandled exceptions shall propagate to clients.	100% structured error responses

2.4 Security

> **Architectural impact.** NFR-11 is the primary reason the API Gateway subsystem exists as a standalone service rather than being embedded in the mobile client. The gateway acts as a secret vault, keeping all third-party API keys server-side and never exposing them in the mobile app bundle, which is inspectable by end users.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-10	Security	All API communication between the mobile client and the backend shall use HTTPS; plaintext HTTP connections shall be rejected.	TLS 1.2+ enforced on all channels
NFR-11	Security	Third-party API keys (Google Places, OpenRouteService) shall never be embedded in the mobile app bundle. All calls to external APIs shall be proxied through the backend API Gateway.	Zero secrets in client bundle

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-12	Security	Firebase Security Rules shall restrict read and write access to trip documents so that only the document owner and explicitly listed collaborators can mutate trip data.	Role-based access enforced at DB layer

2.5 Usability & Maintainability

> **Architectural impact.** NFR-15 codifies the Facade + Strategy design decision as a non-negotiable maintainability requirement. It ensures that future provider migrations (e.g., replacing Overpass with a paid open dataset) do not propagate breaking changes through the React Native UI layer, protecting the product from vendor lock-in.

Req ID	Quality Attr.	Requirement	Metric / Target
NFR-13	Usability	Any primary feature of the app (itinerary builder, map, budget tracker, social feed, profile) shall be reachable from any screen in no more than 3 taps.	Max 3 taps to any primary feature
NFR-14	Maintainability	The codebase shall be organised into independently testable modules that align with subsystem boundaries (itinerary, POI, budget, social, auth); inter-module coupling shall be minimised.	Module fan-out < 5 per module
NFR-15	Maintainability	Swapping the POI provider (e.g., replacing Overpass with a different open dataset) shall require code changes only within the TravelService Facade; no changes to UI screens or API Gateway routes shall be necessary.	Facade fully isolates provider logic

3. Architecturally Significant Requirements — Summary

The following requirements are designated **Architecturally Significant Requirements (ASRs)** because they impose cross-cutting constraints, force the adoption of specific patterns or infrastructure components, or have an outsized influence on overall system structure. Failure to address any ASR would compromise the feasibility, performance, or maintainability of the entire system.

Req ID	Driver	Architectural Impact
FR-03	Offline Persistence & Sync	Mandates AsyncStorage-backed storage and a client-side sync queue. Shapes the Repository pattern and the client data model.
FR-07	API Response Caching	Drives the caching layer inside TravelService Facade. Directly mitigates the risk of exceeding free-tier API quotas.
FR-19	Itinerary Clone	Requires the Social Feed and Itinerary subsystems to share a common trip schema so that cloned documents are immediately editable without data transformation.
FR-20 / FR-21	Real-time Collaboration	Justifies Firebase Realtime Database over a REST polling approach. Requires the Observer pattern with Firebase listeners throughout the Itinerary Engine.
NFR-06	Free-tier Quota Compliance	The central economic constraint of the architecture. Drives both the Facade (central quota control) and the caching Repository (minimise outbound calls).
NFR-07	Offline-first Availability	Elevates offline capability from a post-launch enhancement to a first-class architectural concern, requiring a dedicated sync queue and conflict-resolution strategy.
NFR-11	No Client-side Secrets	Mandates the API Gateway as a standalone subsystem. Without it, third-party API keys would be exposed in the mobile app bundle.
NFR-15	Provider Swappability	Enforces the Facade + Strategy pattern as a hard maintainability requirement, preventing UI components from taking a direct dependency on any specific POI provider.

4. Subsystem Overview

WanderMate is decomposed into **six primary subsystems** and one cross-cutting infrastructure layer. Each subsystem is independently deployable and testable, communicates through well-defined interfaces, and maps directly to one or more functional requirement groups defined in Section 1.

The decomposition follows three guiding principles:

- **Single Responsibility** — each subsystem owns one cohesive slice of domain logic.
- **Low Coupling / High Cohesion** — inter-subsystem communication goes through explicit interface contracts (REST endpoints, Firebase paths, defined service APIs).
- **Replaceability** — the POI provider, the mapping library, and the real-time infrastructure can each be replaced without restructuring other subsystems.

4.1 Subsystem Summary Table

Subsystem	Role & Responsibility	Key Components	Interfaces With
Mobile Client (React Native / Expo)	Entry point for all user interactions. Renders the five bottom-tab screens, manages local state, hosts the Leaflet WebView, and orchestrates calls to the API Gateway and Firebase.	Tab navigator, Leaflet WebView, AsyncStorage cache, Redux/Context state, Firebase SDK client	API Gateway, Firebase Real-time DB, Firebase Auth
API Gateway (Node.js + Express)	Single backend entry point. Proxies and authenticates all third-party API calls, enforces rate limits, applies caching, and serves trip/user/feed data from MongoDB. Keeps all secrets server-side.	Express routes, JWT middleware, rate-limiter, in-memory cache, Mongoose models, Axios clients	Mobile Client, MongoDB, Overpass API, Google Places API, ORS
POI & Routing Service (TravelService Facade)	Encapsulates the dual-API POI strategy (Overpass + Google Places) and the ORS route optimisation call behind a single `TravelService` interface. Implements response caching and provider fallback.	`TravelService` class, `OverpassAdapter`, `PlacesAdapter`, `ORSAdapter`, shared response cache	API Gateway, Overpass API, Google Places API, ORS
Itinerary & Budget Engine	Manages the full lifecycle of trip documents: CRUD, drag-and-drop reordering (Command pattern, undo/redo), expense logging, split calculation, and offline sync queue.	`TripRepository`, `BudgetRepository`, `CommandManager`, `SyncQueue`, Firebase write listeners	API Gateway, Firebase Real-time DB, AsyncStorage

Subsystem	Role & Responsibility	Key Components	Interfaces With
Social Feed & Community Service	Handles trip publishing, follower graph management, like counts, feed pagination, and one-tap itinerary cloning. Publishes update events to Firebase for real-time card refresh.	`FeedRepository`, `CloneService`, `FollowGraph`, Firebase feed listeners, feed card components	API Gateway, Firebase Realtime DB, Itinerary Engine
Auth & User Profile	Manages user identity via Firebase Authentication (Google Sign-In), stores user profile data in MongoDB, and enforces Firebase Security Rules for per-trip access control.	Firebase Auth SDK, Google Sign-In provider, `UserRepository`, Firebase Security Rules	Firebase Auth, API Gateway, Firebase Realtime DB
Real-time Sync Infrastructure (Firebase — cross-cutting)	Cross-cutting layer providing event-driven data propagation for collaborative itinerary edits and social feed updates. All reactive subsystems register Firebase listeners rather than polling.	Firebase Realtime Database, Security Rules, connection-state listeners, offline queue	Itinerary Engine, Social Feed Service, Mobile Client

4.2 Detailed Subsystem Descriptions

4.2.1 Mobile Client (React Native / Expo)

The mobile client is the sole user-facing component of WanderMate. It is built with React Native using the Expo managed workflow, targeting Android. Navigation is provided by a bottom tab bar with five primary screens: **Trips**, **Explore**, **Feed**, **Budget**, and **Profile**.

The interactive map is rendered by embedding Leaflet inside a `WebView` component with a bidirectional JavaScript bridge for passing itinerary data in and receiving user map interactions out. AsyncStorage provides an offline-first data layer: when the device is offline the client reads from and writes to local storage, queuing mutations for server sync on reconnect. All outbound requests carry a Firebase JWT obtained at login; the API Gateway validates this token on every request.

> **Key patterns:** Observer (Firebase listeners for live updates), Command (undo/redo via CommandManager), Repository (AsyncStorage abstraction).
>
> **Satisfies:** FR-01 to FR-04, FR-09 to FR-11, NFR-07, NFR-13.

4.2.2 API Gateway (Node.js + Express)

The API Gateway is the single backend service in WanderMate's architecture. Its primary architectural function is to act as a *secret vault* — all third-party API keys (Google Places, ORS) reside here as environment variables and are never transmitted to or stored on the mobile client.

In addition to secret management, the gateway performs: Firebase JWT verification via middleware, per-user rate limiting on external API proxying, an in-memory (or Redis) cache pass to reduce duplicate outbound calls, and CRUD REST endpoints for trips, users, expenses, and feed posts backed by MongoDB (Mongoose ODM). The gateway is stateless and horizontally scalable behind a load balancer.

> **Key patterns:** Facade (single entry point abstracting all third-party APIs), Repository (Mongoose models).
> **Satisfies:** NFR-04, NFR-09, NFR-10, NFR-11.

4.2.3 POI & Routing Service (TravelService Facade)

This module implements the **Facade** and **Strategy** patterns. `TravelService` exposes three methods to the gateway layer: `searchPOI(query, category)`, `getPlaceDetails(placeId)`, and `getRoute(waypoints)`. Internally, three adapters implement a shared `IPoiAdapter` interface:

- **OverpassAdapter** — queries the Overpass API for OpenStreetMap data (free, unlimited, used for search).
- **PlacesAdapter** — enriches search results with Google Places ratings, photos, and opening hours (quota-sensitive: 5 000 req/month).
- **ORSAdapter** — requests optimised multi-stop routes from OpenRouteService (2 000 req/day).

A shared response cache sits in front of all three adapters. If Google Places is unavailable at runtime, `TravelService` falls back to returning Overpass-only results, satisfying NFR-08 without any change to the calling code.

> **Key patterns:** Facade (`TravelService` wraps three APIs), Strategy (swappable `IPoiAdapter` implementations).
> **Satisfies:** FR-05 to FR-08, FR-09 to FR-11, NFR-01, NFR-06, NFR-08, NFR-15.

4.2.4 Itinerary & Budget Engine

This subsystem owns the core domain logic of WanderMate. `TripRepository` and `BudgetRepository` abstract MongoDB document access from the business layer, implementing the **Repository** pattern so that the storage backend can be replaced independently of business rules.

All itinerary mutations (add stop, remove day, reorder stop) are modelled as **Command** objects with `execute()` and `undo()` methods. A `CommandManager` maintains an ordered stack, enabling multi-level undo/redo without bespoke state management in the UI layer (satisfies FR-04).

A `SyncQueue` persists unsent mutations to AsyncStorage when the device is offline and flushes them in timestamp order on reconnect. Collaborative edits are broadcast through Firebase Realtime Database; this subsystem both writes change events to

Firebase on local mutation and subscribes to remote events via `onValue` listeners, implementing the **Observer** pattern.

> **Key patterns:** Repository, Command (undo/redo), Observer (Firebase listeners).
> **Satisfies:** FR-01 to FR-04, FR-12 to FR-15, FR-20 to FR-21, NFR-03, NFR-07.

4.2.5 Social Feed & Community Service

The social feed subsystem manages the public layer of WanderMate. When a user publishes a trip, `PublishService` serialises a feed post (route thumbnail, metadata snapshot, and a read-only copy of the itinerary) and writes it atomically to both MongoDB (as the authoritative document) and Firebase Realtime Database (to broadcast to follower feed listeners). This dual-write ensures feed cards appear in real time on all connected followers' screens without polling.

`CloneService` performs a deep copy of a published trip document, reassigns the cloning user as the new owner, clears the `published` flag, and writes the copy as a new MongoDB document in a single atomic operation. This prevents partial clones in the event of a mid-operation failure and ensures the cloned trip is immediately editable (satisfies FR-19).

The follower graph (`FollowGraph`) is stored as an adjacency list in MongoDB, with denormalised display names and avatar URLs cached in Firebase for fast feed card rendering.

> **Key patterns:** Observer (Firebase feed listeners), Repository (`FeedRepository`).
> **Satisfies:** FR-16 to FR-19, NFR-02, NFR-03.

4.2.6 Authentication & User Profile

Identity management is delegated entirely to Firebase Authentication, using the Google Sign-In OAuth 2.0 provider. On first successful sign-in, the API Gateway creates a corresponding `User` document in MongoDB containing the user's display name, avatar URL, follower/following ID lists, and references to their published trips.

Firebase Security Rules are configured to match trip document ownership and the `collaborators` array, ensuring that only the owner and explicitly authorised collaborators can read or write sensitive trip data in Firebase Realtime Database (satisfies NFR-12). Profile updates flow through the API Gateway to MongoDB; display name and avatar URL are also denormalised into Firebase to avoid expensive joins at feed card render time.

> **Key patterns:** Repository (`UserRepository`).
> **Satisfies:** FR-22, FR-23, NFR-10, NFR-11, NFR-12.

4.2.7 Real-time Sync Infrastructure (Firebase — Cross-cutting)

Firebase Realtime Database serves as the event bus for WanderMate's two real-time channels:

- **Collaborative itinerary editing** — changes by any collaborator are written to a dedicated Firebase path and broadcast to all listeners within 500 ms (NFR-03).
- **Social feed updates** — new published posts are written to a `/feed/{userId}` path; followers register `onChildAdded` listeners for real-time card injection.

This layer is intentionally kept thin. It carries events and deltas only; full document state remains authoritative in MongoDB. This architectural decision prevents Firebase from becoming a second source of truth for complex relational queries (follower graphs, expense aggregations), which it is not optimised for. Firebase's managed connection pooling, automatic reconnection, and built-in offline queue remove the need for a custom WebSocket server.

> **Key patterns:** Observer (all reactive subsystems attach Firebase listeners).
> **Satisfies:** FR-15, FR-20, FR-21, NFR-03, NFR-07.

5. Subsystem Interaction Flows

The following five runtime data flows describe how subsystems collaborate to deliver WanderMate's core behaviours. These flows directly trace back to the ASRs identified in Section 3.

#	Flow	Step-by-step Description
1	POI & Route Query	Mobile Client → API Gateway (HTTPS + JWT) → TravelService Facade checks cache; on miss, calls OverpassAdapter (search) + PlacesAdapter (enrich) → cached result returned to client → ORSAdapter computes route polyline → Leaflet WebView renders map overlay.
2	Collaborative Edit	Collaborator A edits stop on Mobile Client → Itinerary Engine executes Command, writes delta to MongoDB via API Gateway, and simultaneously writes event to Firebase → Firebase broadcasts to all registered listeners → Collaborator B's Mobile Client receives update and re-renders itinerary.
3	Feed Publish	User taps *Publish* → Social Feed Service serialises feed post, writes atomically to MongoDB + Firebase `/feed/` path → Firebase propagates `onChildAdded` event to all follower clients → feed cards update in real time without polling.
4	Itinerary Clone	User taps *Clone* on a feed card → CloneService deep-copies trip document in MongoDB, assigns new owner, clears `published` flag, commits atomically → new trip appears in user's Trips tab immediately.
5	Offline Sync	Device goes offline → Itinerary Engine routes mutations to AsyncStorage SyncQueue → device reconnects → SyncQueue flushes ordered mutations to API Gateway → MongoDB updated → Firebase receives resolved state → remote collaborators' views converge.